MASTER'S MODULE · AGENTIC AI · WINTER SEMESTER

Frameworks vs. Plain Code

The build-or-adopt decision: what agent frameworks actually provide, what they quietly cost, and why a hundred lines of plain Python stays on the table.

Unit 05 · Part 2 of 2 · Session 07 · Week 7
Prof. Dr. Hauke Schramm · Department of Computer Science · Kiel University

Today's agenda

1. **Where we are** patterns chosen in Session 06 — now: who writes the loop?
2. **The landscape** LangGraph, CrewAI, AutoGen, smolagents, vendor SDKs — as concepts
3. **The ledger** what frameworks buy you vs. what they cost
4. **Start simple** Anthropic's guidance and a 100-line plain-code baseline
5. **Same agent, twice** plain Python vs. a LangGraph state graph, line by line
6. **Deciding & exiting** a checklist, migration strategies, and this week's lab

Where we are

Session 06 chose the orchestration pattern; today we choose who implements it.

Last week — ReAct, plan-and-execute, DAGs — control-flow shapes on the predictability-flexibility axis.

Open question — every pattern can be written by hand or assembled from a framework's parts.

Today — the architecture decision: adopt a framework, use a thin SDK, or own the loop yourself.

The method — build one tool-using agent both ways — plain Python, then a state graph — and compare what we can see, debug, and keep; Lab 07 repeats this with measurements.

The stance — neither camp wins by default — this is an engineering trade, not a fashion choice.

ONE IDEA TODAY

A framework is code you run but did not write — including prompts. Adopt one for what it provides, and only with an exit plan.

1

The framework landscape

Five families of abstractions, surveyed as concepts

The landscape at a glance

Five families — what each one believes an agent is.

LangGraph

The agent as an explicit state graph: typed state with reducers, nodes, conditional edges; checkpointing and interrupts built in. Aimed at complex control flow.

CrewAI

Role-played teams: agents with role, goal and backstory work through tasks, sequentially or under a manager. Fast to demo, heavily opinionated.

AutoGen, then MAF

AutoGen (historical line): everything is a conversation. Its successor, Microsoft Agent Framework (1.0, 2026), adds typed agents, threads and graph-based workflows with checkpoints.

smolagents

Hugging Face minimalism, ~1,000 core lines. CodeAgent acts by writing Python (Wang et al., 2024); ToolCallingAgent uses JSON tool calls.

OpenAI Agents SDK

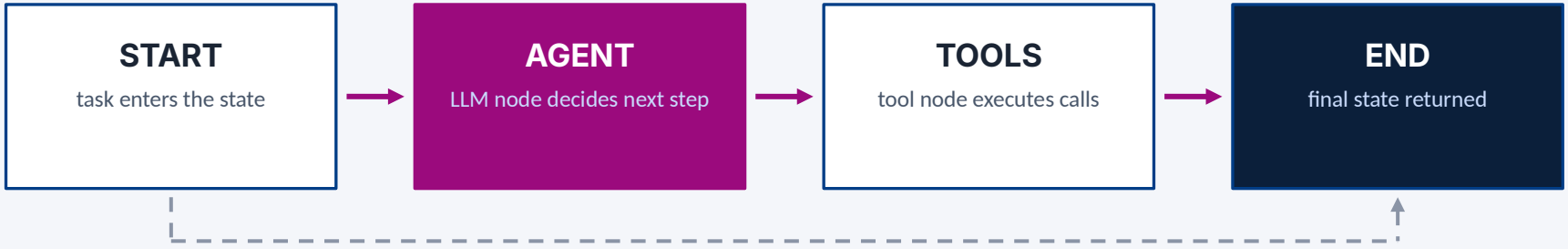
A thin vendor loop: agents, handoffs, guardrails, sessions, tracing included. Production successor to the experimental Swarm.

Claude Agent SDK

The loop behind Claude Code, packaged: tools, hooks, subagents, permissions, MCP. An opinionated harness, not a graph builder.

LangGraph: the agent as a state graph

Our S02 while-loop, re-expressed as nodes, edges, and a checkpointer.



edge tools -> agent: the loop, made explicit

- State is a typed, shared object; each node returns an update, and a reducer (e.g. add_messages) defines how parallel updates merge.
- A conditional edge replaces the if-statement: tool call goes to TOOLS, plain text to END.
- A checkpointer persists state at superstep boundaries — resume, replay, pauses; a tool's external side effect is not rolled back.

Five families, one table

Concepts, not APIs — syntax churns, the abstractions persist.

Family	Core abstraction	Control flow	Persistence
LangGraph	typed state graph with reducers	explicit edges, conditional routing	checkpointer (SQLite, Postgres)
CrewAI	roles, tasks, crew	sequential or manager-led process	limited; memory add-ons
AutoGen (historical)	conversing agents	emergent from dialogue	conversation history
MS Agent Framework	typed agents, threads, workflows	explicit graph workflows, handoffs	threads; workflow checkpoints
smolagents	CodeAgent or ToolCallingAgent	Python steps or JSON tool calls	minimal by design
Vendor SDKs	agent + tools + handoffs	provider-shaped loop	sessions, managed state

All of them wrap the same loop from Session 02 — they differ in what they make explicit.

Vendor SDKs: the thin middle ground

Less machinery than LangGraph, more than raw API calls — with strings attached.

OpenAI Agents SDK — agents, handoffs between them, guardrails, sessions; tracing wired to OpenAI's dashboard (2025).

Claude Agent SDK — ships Claude Code's production loop: context compaction, hooks, subagents, permission gates.

What they share — a fixed loop with tool execution, provider-tuned defaults, first-party tracing and session storage — no graph engine.

The catch — the SDK is also a distribution channel — defaults, tracing and sessions pull you toward one vendor.

Fit — strong when you are committed to that provider anyway and want the harness, not a graph engine.

PRACTICAL ADVICE

Before adopting an SDK, read its loop, its default prompts and its tool-execution path in the source. Then you know what you can replace — and what you cannot.

2

What you buy, what you pay

The ledger of abstraction, both columns

What a framework actually buys you

Six services you would otherwise build, test and maintain yourself.

State management

Typed state schemas replace ad-hoc dicts; parallel branches merge safely only where you declare a reducer — the type alone does not.

Retries & error policy

Backoff, timeouts and fallback models declared per node — instead of try-except blocks copy-pasted around every call.

Persistence

Checkpoints at superstep boundaries: kill the process, resume the run, rewind and replay. A tool's external write is not undone — resume must reconcile it.

Tracing hooks

Every LLM and tool call emits structured spans; LangSmith, Langfuse or OpenTelemetry attach without touching agent logic.

Streaming plumbing

Token and event streams piped from nested calls to your UI — tedious, bug-prone code nobody wants to own.

Human-in-the-loop

Interrupt before a risky node, wait hours for approval, resume from the checkpoint — built on the persistence layer.

What it costs you

The costs are real, deferred, and mostly invisible at demo time.

Abstraction opacity — prompts and control flow you never wrote execute in your name; behaviour is partly folklore.

Debugging through layers — a stack trace now crosses graph runtime, callback managers and serializers before reaching your code.

Version churn — LangChain went 0.1 to 0.2 to 0.3 to 1.0 in under two years — 1.0 (Oct 2025) even replaced create_react_agent with create_agent.

Lock-in — state schemas, checkpointer formats and tracing become load-bearing; leaving means rewriting.

Hidden prompt magic — role scaffolding and injected instructions shift between versions — silently changing behaviour.

TIMING ASYMMETRY

Benefits arrive on day one, at the demo. Costs arrive in month three, in production, at 2 a.m. — evaluate both on the same timescale.

The hidden prompt problem

Your agent's behaviour can change on pip install -U — without you editing a single line.

Frameworks embed prompt templates, role scaffolding and stop heuristics inside library code. A version bump rewrites text the model reads on every call — the diff is in someone else's repository.

3

Start simple: a 100-line baseline

The same agent twice — plain Python, then LangGraph

Anthropic's advice: start simple

Building Effective Agents (2024) — written by people who ship agents for a living.

The principle — find the simplest solution possible; add complexity only when it demonstrably improves outcomes.

Start at the API — use LLM APIs directly first — most patterns from Session 06 are a few lines of code.

If you adopt one — understand what is under the hood; wrong assumptions about hidden layers are a major error source.

Workflows first — fixed, composable workflows cover most production cases before a dynamic agent is justified.

Not anti-framework — the advice is sequencing: earn the complexity, then adopt tools that manage it.

MATURITY INVERSION

Beginners reach for frameworks to avoid the loop; experienced teams write the loop first, then adopt a framework deliberately — or not at all.

Plain code 1/2 — tools and retries

The S02 loop grown up: a tool registry and bounded retries for transient failures only (cloud-API example).

agent.py (excerpt 1 of 2, ~100 lines total) -- cloud example

```
# the S02 mini-agent, hardened for real runs
TOOLS = {"web_search": web_search, "save_report": save_report}
client = OpenAI(max_retries=0)     # one retry budget: ours
TRANSIENT = (RateLimitError, APITimeoutError,
             APIConnectionError, InternalServerError)

def call_llm(messages, retries=3):
    for attempt in range(retries):
        try:
            return client.chat.completions.create(
                model=MODEL, messages=messages,
                tools=SCHEMAS, timeout=60)
        except TRANSIENT as e:    # 4xx bad request: no retry
            if attempt == retries - 1: break  # no final sleep
            time.sleep(retry_after(e) or 2 ** attempt)
    raise RuntimeError("request failed after bounded retries")
```

Plain code 2/2 — loop and snapshot

Best-effort snapshot at step boundaries; durable, exactly-once resume deliberately omitted.

agent.py (excerpt 2 of 2) -- cloud example

```
def save(state):                  # best-effort snapshot
    tmp = Path("run.json.tmp")
    tmp.write_text(json.dumps(state))
    os.replace(tmp, "run.json")  # atomic swap, no torn file

state = load("run.json") or fresh_state(task)
while state["step"] < MAX_STEPS:
    msg = call_llm(state["messages"]).choices[0].message
    state["messages"].append(to_dict(msg))
    if not msg.tool_calls:
        print(msg.content); break          # final answer
    for c in msg.tool_calls:               # side effects!
        state["messages"].append(run_tool(c))
    state["step"] += 1
    save(state)      # at a step boundary: no dangling call
# a mid-step crash re-runs its tools on resume: idempotent?
```

LangGraph 1/2 — state and nodes

Typed state, one reasoning node, one tool node. ToolNode only executes; bind_tools lets the model see the schemas.

graph_agent.py (1 of 2)

```
import sqlite3
from typing import Annotated, TypedDict
from langgraph.graph import StateGraph, START
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode, tools_condition
from langgraph.checkpoint.sqlite import SqliteSaver

class State(TypedDict):           # typed shared state
    messages: Annotated[list, add_messages]   # reducer
tools = [web_search, save_report]  # same schemas as plain
llm_with_tools = llm.bind_tools(tools)  # model gets schemas
def agent(state: State):                # one reasoning step
    reply = llm_with_tools.invoke(state["messages"])
    return {"messages": [reply]}
g = StateGraph(State); g.add_node("agent", agent)
g.add_node("tools", ToolNode(tools))    # executes calls
```

LangGraph 2/2 — edges and persistence

Control flow as data; checkpointing by configuration. The interrupt only pauses — the approval decision is still your code.

graph_agent.py (2 of 2)

```
g.add_edge(START, "agent")
g.add_conditional_edges("agent", tools_condition)
g.add_edge("tools", "agent")        # the loop is an edge
conn = sqlite3.connect("runs.db", check_same_thread=False)
app = g.compile(checkpointer=SqliteSaver(conn),
                interrupt_before=["tools"])  # pause only
cfg = {"configurable": {"thread_id": "report-42"},
       "recursion_limit": 2 * MAX_STEPS}  # counts supersteps
app.invoke({"messages": [("user", task)]}, cfg)
while (snap := app.get_state(cfg)).next:    # paused?
    calls = snap.values["messages"][-1].tool_calls
    if not approve(calls): break    # rejected: stays paused
    app.invoke(None, cfg)           # approved: run one step
# process killed? invoke(None, cfg) resumes the thread --
# after you reconcile tool side effects already executed
```

Same behaviour, different ownership

Same task, tool schemas and step budget; retries and stop rules must be configured on both sides. The difference is who owns what.

Plain code (~100 lines)

- Every behaviour is visible in your repo
- Debugging means reading your own code
- Persistence exactly as good as you built
- Streaming, tracing, parallelism: your backlog
- Fewer orchestration dependencies, no hidden prompts

LangGraph (~40 lines)

- Checkpoints, interrupts, replay by configuration
- Control flow declared as inspectable data
- Tracing and streaming attach per node
- Behaviour partly defined by library internals
- Upgrades can change semantics under you

4

Deciding — and exiting cleanly

A checklist, migration strategies, and the lab

A decision checklist

First the must-haves (e.g. durable resume, data residency) — a single hard requirement can decide. Then weigh the remaining rows.

Factor	Leans plain code	Leans framework
Team	1-3 devs, strong Python	larger team, shared vocabulary needed
Lifetime	prototype, paper, one semester	multi-year product with handovers
Control flow	one loop, few branches	many branches, parallelism, approval gates
Observability	logs and print suffice	per-step traces, replay, audit trail
Persistence	best-effort restart acceptable	durable resume for long-running jobs
Maintenance	original authors stay close	staff turnover; docs beat tribal knowledge

Rows are not equal: weight them for your project, and let any unmet must-have override the count.

Exit strategies: design for divorce

Adopt any framework as if you will leave it — because one day you will.

Ports and adapters

Your agent logic depends on interfaces you define; the framework lives behind an adapter. Swapping it touches one module, not the whole system.

Own every prompt

Keep all prompt text in your repository, injected explicitly. Never ship a framework's default prompts — they change between versions and you cannot diff them.

Own the message log

Persist the message log in your own versioned event format, with adapters to and from each provider's wire format. Neither framework state objects nor one vendor's message schema is portable.

Evals as a safety net

A behaviour test suite over recorded tasks lets you swap implementations and verify equivalence — migration becomes refactoring, not rewriting.

Lab 07: build it both ways

Gap-fill notebook, Parts A-F, 90-120 min; report tasks R1-R4.

Setup — local Ollama (qwen2.5:7b), no API keys; a search-read-write task over an offline corpus in data/.

Variant A — grow the Lab 02 loop to ~100 lines: tool registry, exponential-backoff retries, JSON checkpoint and resume.

Variant B — no pip install — rebuild LangGraph's API shape in plain Python: add_messages reducer, nodes, conditional edges, SQLite checkpointer with thread_id.

Measure — lines of code, dependency count, time to diagnose an injected description-mismatch bug, the kill test.

Report — R1 LOC critique, R2 rewind-and-replay code, R3 the CTO memo defended with the checklist, R4 exit-strategy audit.

THE POINT

Not to crown a winner — to make the trade-off concrete enough that your next architecture decision is a judgement, not a reflex.

TAKEAWAYS

What you should remember

- **Five families —** graphs (LangGraph), role teams (CrewAI), conversations (AutoGen, now MAF workflows), code- or tool-calling agents (smolagents), vendor SDKs.
- **Frameworks sell services —** state, retries, persistence, tracing, streaming, human-in-the-loop — real engineering you would otherwise own.
- **Costs are deferred —** opacity, layered debugging, version churn, lock-in, hidden prompts — invisible at the demo, expensive in production.
- **Start simple —** Anthropic's guidance: direct API calls first; add a framework only when it demonstrably earns its place.
- **100 lines for the demonstrator —** retries and a best-effort snapshot fit in plain Python; durable resume, replay and interrupts are what a checkpointer buys — at the price of ownership.
- **Design for divorce —** own your prompts and your message log, hide frameworks behind adapters, keep an eval suite as the exit door.

NEXT SESSION Memory — short-term context, long-term stores, and what agents should remember.

Reading & references

- Anthropic (2024). Building Effective Agents. anthropic.com/engineering/building-effective-agents.
  The 'start simple' argument and the workflow patterns this unit builds on. Read in full.
- Wang, X. et al. (2024). Executable Code Actions Elicit Better LLM Agents. ICML 2024. arXiv:2402.01030.
  The CodeAct result behind smolagents: code actions beat JSON tool calls on their benchmarks.
- Wu, Q. et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155.
  The conversation-centric design; skim for the abstraction, not the API.
- LangChain (2025). LangGraph documentation: concepts — state, persistence, human-in-the-loop. docs.langchain.com/oss/python/langgraph (LangGraph 1.2, 2026).
  Read the concept pages; they explain the checkpointer model used today.
- OpenAI (2025). OpenAI Agents SDK documentation. openai.github.io/openai-agents-python.
  Agents, handoffs, guardrails, sessions — read the loop and defaults before adopting.
- Anthropic (2025). Building Agents with the Claude Agent SDK. anthropic.com/engineering.
  How Claude Code's loop is packaged: hooks, subagents, permissions, compaction.
- Hugging Face (2025). smolagents documentation. huggingface.co/docs/smolagents.
  The minimalist counter-position: ~1,000 core lines, code-acting agents.